Doc. no: P0286R0
Date: 2016-02-14
Audience: Library Evolution
Reply-To: Christopher Kohlhoff <chris@kohlhoff.com>

A networking library extension to support co_await-based coroutines

1. Introduction

This paper outlines a pure extension to the draft Networking Technical Specification to add support for `co_await`-based coroutines. This extension allows us to leverage coroutines to write asynchronous code in a synchronous style, as in:

```
awaitable<void> echo(tcp::socket socket, await_context ctx)
{
  try
  {
    for (;;)
    {
      char data[128];
      std::size_t n = co_await socket.async_read_some(net::buffer(data), ctx);
      co_await async_write(socket, net::buffer(data, n), ctx);
    }
  }
  catch (std::exception& e)
  {
    std::cerr << "echo Exception: " << e.what() << std::endl;
  }
}
```

The design presented in this paper reflects the view that, when using coroutines to compose asynchronous operations, coroutines must be considered in conjunction with executors. Typical networking programs consist of multiple threads of execution (whether implemented using coroutines or as simple chains of callbacks). Indeed, one of the motivations for using coroutines and asynchronous operations is greater control over scheduling than that provided by the OS's thread scheduler. This control allows for both better performance and simplified programming.

Consequently, the design presented below has the following features:

- The execution properties of the current coroutine-based thread of execution are explicitly represented by an `await_context` object. This object is a completion token, and when passed to an asynchronous operation causes the operation to "block" the current coroutine in a synchronous-like manner.
- To mirror the semantics of normal functions, coroutines may be composed into a "stack", and all coroutines in the stack behave as though part of the same thread of execution.
- New coroutine-based threads of execution are explicitly launched using a `spawn` function. This function also allows the user to specify the execution properties of the new thread of execution.

2. Reference implementation

An implementation of this proposal text may be found in a branch of the variant of Asio that stands alone from Boost. This branch is available at *https://github.com/chriskohlhoff/asio/tree/co_await*. It has been tested with Microsoft Visual Studio 2015 Update 1, and depends specifically on the version of the proposed coroutine functionality delivered with that compiler.

3. Examples

3.1. Basic use

To begin, we will examine a simple TCP server that echoes back any data it receives. The main function is as follows:

```
int main()
{
  try
  {
    net::io_context io_context;
    spawn(io_context, listener, detached);
```

```
    io_context.run();
  }
  catch (std::exception& e)
  {
    std::cerr << "Exception: " << e.what() << std::endl;
  }
}
```

Here, the call to the function `spawn`:

```
spawn(io_context, listener, detached);
```

launches a coroutine as a new thread of execution. The first argument specifies that this new thread of execution will be scheduled by the `io_context`. The entry point for this new thread of execution is the function `listener`, which we will see below. The final argument, `detached`, is a special completion token that tells `spawn` that we are not interested in the result of the coroutine.

The `listener` is a free function:

```
awaitable<void> listener(await_context ctx)
{
  tcp::acceptor acceptor(ctx.get_executor().context(), {tcp::v4(), 55555});
  for (;;)
  {
    spawn(acceptor.get_executor(), echo,
        co_await acceptor.async_accept(ctx), detached);
  }
}
```

The `listener` function returns an `awaitable<void>`. This indicates that it must either be the entry point of a new thread of execution, or itself be `co_await`-ed.

The `listener` function also accepts an `await_context` as its parameter. This parameter represents the context in which the coroutine is executing, and is passed as a completion token to any asynchronous operations called by the coroutine, such as:

```
co_await acceptor.async_accept(ctx)
```

When the `ctx` completion token is passed to an asynchronous operation, that operation's initiating function returns an `awaitable<T>`. We must apply the `co_await` keyword to this return value to suspend the coroutine.

In this listener, private state (such as `acceptor`) may simply be declared as a stack-based variable. As each new connection is accepted, the listener spawns a new, detached thread of execution to handle the incoming client:

```
spawn(acceptor.get_executor(), echo,
    co_await acceptor.async_accept(ctx), detached);
```

The first argument to `spawn` specifies that the new thread of execution will be scheduled using the acceptor's `io_context`. This is the `io_context` object that we created in `main`. In the case where multiple threads are running the `io_context`, this would allow the new thread of execution to execute concurrently. This is a safe choice only if the new thread of execution is truly independent and does not access shared data structures. (Note that, in this example, only the main thread runs the `io_context` and so all coroutines will be scheduled in a single thread in any case.)

The entry point for the new thread of execution is the `echo` function, and this time we are passing it the result of the `async_accept` operation. The `echo` function accepts this result in its parameter list:

```
awaitable<void> echo(tcp::socket socket, await_context ctx)
{
  try
  {
    for (;;)
    {
      char data[128];
      std::size_t n = co_await socket.async_read_some(net::buffer(data), ctx);
      co_await async_write(socket, net::buffer(data, n), ctx);
    }
  }
  catch (std::exception& e)
  {
    std::cerr << "echo Exception: " << e.what() << std::endl;
```

```
  }
}
```

As with the `listener`, private state such as the `data` buffer may simply be specified as stack variable in the coroutine. We pass the `ctx` completion token to the asynchronous operations, and `co_await` the `awaitable<T>` objects that they return. Any errors are reported as exceptions, so we catch these within the coroutine to prevent them from escaping to the `main` function.

3.2. Refactoring

Just as with normal, synchronous function calls, when using coroutines we wish to be able to refactor a sequence of code into its own function. When doing so, it is vital for ensuring program correctness that the refactored code execute in the same thread of execution, and have the same executor properties as its caller.

For example, lets us say we wish to refactor the `echo` function above so that a single `async_read_some`/`async_write` pair is in its own `echo_once` function:

```
awaitable<void> echo_once(tcp::socket& socket, await_context ctx)
{
  char data[128];
  std::size_t n = co_await socket.async_read_some(net::buffer(data), ctx);
  co_await net::async_write(socket, net::buffer(data, n), ctx);
}
```

This function is then called from `echo` as follows:

```
awaitable<void> echo(tcp::socket socket, await_context ctx)
{
  try
  {
    for (;;)
    {
      co_await echo_once(socket, ctx);
    }
  }
  catch (std::exception& e)
  {
    std::cerr << "echo Exception: " << e.what() << std::endl;
  }
}
```

By passing the `ctx` variable to `echo_once` we ensure that it is scheduled using the same executor. Furthermore, the caller applies `co_await` to the `awaitable<T>` produced by `echo_once`, guaranteeing that the `echo` function does not resume until the callee is complete. These two attributes combine to ensure that the `echo_once` function behaves as though part of the same thread of execution as `echo`.

3.3. Coordinating related threads of execution

The echo server shown above is a trivially asynchronous program in that:

- the protocol is half-duplex, so there is only one chain of operations associated with the connection; and
- the connections do not share data or otherwise interact with any others in the program.

More typically, connection handling involves a number of concurrent threads of execution, such as:

- one to read inbound data;
- one to write queued outbound data;
- one to manage timeouts or heartbeats; and
- short-lived threads of execution representing messages from other actors.

As an example, consider a simple chat server where multiple connections share a chat room. Any message sent by a participant to the room is relayed by the server to all participants.

```
class chat_session
  : public chat_participant,
    public std::enable_shared_from_this<chat_session>
```

The `chat_session` class is comprised of multiple coroutine-based threads of execution. We want the session to exist for as long as there is client activity, so we use `std::enable_shared_from_this` to keep the `chat_session` object alive for as

long as its constituent coroutines.

```
{
  tcp::socket socket_;
  net::steady_timer timer_;
  chat_room& room_;
  std::deque<std::string> write_msgs_;
  net::strand<net::io_context::executor_type> strand_;
```

The `chat_session` class uses a strand to coordinate the threads of execution and ensure that they do not execute concurrently.

```
public:
  chat_session(tcp::socket socket, chat_room& room)
    : socket_(std::move(socket)),
      timer_(socket_.get_executor().context()),
      room_(room),
      strand_(socket_.get_executor())
  {
    timer_.expires_at(std::chrono::steady_clock::time_point::max());
  }

  void start()
  {
    room_.join(shared_from_this());
    spawn(strand_, &chat_session::reader, shared_from_this(), detached);
    spawn(strand_, &chat_session::writer, shared_from_this(), detached);
  }
```

The strand is specified as the executor when launching the two threads of execution using `spawn`.

```
  void deliver(const std::string& msg)
  {
    strand_.dispatch(
        [this, self=shared_from_this(), msg]
        {
          write_msgs_.push_back(msg);
          timer_.cancel_one();
        });
  }
```

The `deliver` function uses a short-lived non-coroutine-based thread of execution to add new messages to the outbound write queue.

```
private:
  awaitable<void> reader(await_context ctx)
  {
    try
    {
      for (std::string read_msg;;)
      {
        std::size_t n = co_await net::async_read_until(socket_,
            net::dynamic_buffer(read_msg, 1024), "\n", ctx);

        room_.deliver(read_msg.substr(0, n));
        read_msg.erase(0, n);
      }
    }
    catch (std::exception&)
    {
      stop();
    }
  }

  awaitable<void> writer(await_context ctx)
  {
    try
    {
      while (socket_.is_open())
      {
        if (write_msgs_.empty())
```

```
          {
            std::error_code ec;
            co_await timer_.async_wait(redirect_error(ctx, ec));
```

By default, passing an `await_context` to an asynchronous operation will cause errors to be reported via exception. In this case we handle the error as an expected case, so we use the `redirect_error` completion token to capture the error into an `error_code`.

```
          }
          else
          {
            co_await net::async_write(socket_,
                net::buffer(write_msgs_.front()), ctx);
            write_msgs_.pop_front();
          }
        }
      }
      catch (std::exception&)
      {
        stop();
      }
    }

    void stop()
    {
      room_.leave(shared_from_this());
      socket_.close();
      timer_.cancel();
    }
  };
```

4. Summary of library facilities

This paper proposes the following extensions to the Networking Technical Specification to add support for `co_await`-based coroutines.

4.1. Class template `awaitable`

```
template<class T> awaitable;
```

Class template `awaitable` represents the return type of an asynchronous operation when used with coroutines, or of a coroutine function that composes asynchronous operations. The `awaitable<T>` class satisfies the Awaitable type requirements.

An `awaitable<T>` can be consumed by at most one `co_await` keyword.

4.2. Class template `basic_unsynchronized_await_context`

```
template<class Executor> class basic_unsynchronized_await_context;
```

Class template `basic_unsynchronized_await_context` is a completion token type that causes asynchronous operations to produce an `awaitable<T>` as their initiating function return type.

`basic_unsynchronized_await_context<Executor>` class introduces no synchronization on top of the underlying `Executor` object. It requires an executor that provides mutual exclusion semantics. This minimizes the overhead of coroutines when executing on a single threaded `io_context`, since it is implicitly a mutual exclusion executor.

4.3. Template alias `basic_await_context`

```
template<class Executor>
using basic_await_context = basic_unsynchronized_await_context<strand<Executor>>;
```

`basic_await_context` is a template alias that addresses the common use case of coordinating coroutine execution in a multithreaded context (such as a thread pool). It uses a `strand<>` to provide the requisite mutual exclusion semantics.

4.4. Typedef `await_context`

```
typedef basic_await_context<executor> await_context;
```

This typedef uses the `basic_await_context` template with the polymorphic executor wrapper. This maximizes ease of use, particularly when calling coroutine functions across module boundaries, with some runtime cost.

4.5. Function template `spawn`

```
template<class Executor, class F, class Arg1, ..., class ArgN, class CompletionToken>
  DEDUCED spawn(const Executor& ex, F&& f, Arg1&& arg1, ..., ArgN&& argN,
    CompletionToken&& token);

template<class ExecutionContext, class F, class Arg1, ..., class ArgN, class CompletionToken>
  DEDUCED spawn(ExecutionContext& ctx, F&& f, Arg1&& arg1, ..., ArgN&& argN,
    CompletionToken&& token);

template<class Executor, class F, class Arg1, ..., class ArgN, class CompletionToken>
  DEDUCED spawn(const basic_unsynchronized_await_context<Executor>& ctx,
    F&& f, Arg1&& arg1, ..., ArgN&& argN, CompletionToken&& token);
```

The function template `spawn` is used to launch a new coroutine-based thread of execution.

The first argument determines the executor to be used for scheduling the coroutine. In the case of the final overload, the new coroutine inherits the executor of the specified `basic_unsynchronized_await_context`. (This final overload is provided as a convenience for launching related coroutines that should not be scheduled concurrently.)

These overloads shall not participate in function overload resolution unless the return type of `f(arg1, ..., argN, basic_unsynchronized_await_context<Executor>)` is an `awaitable<T>` for some type `T`.

Note that the function `spawn` meets the requirements of an asynchronous operation, which means that we can pass any completion token type to it. In the examples above, we use the `detached` completion token which is defined in this proposal, but other options include plain callbacks:

```
awaitable<int> my_coroutine(await_context ctx);
// ...
spawn(my_executor, my_coroutine, [](int result) { ... });
```

or the `use_future` completion token:

```
awaitable<int> my_coroutine(await_context ctx);
// ...
std::future<int> f = spawn(my_executor, my_coroutine, std::experimental::use_future);
```

4.6. Class `detached_t`

```
class detached_t { };
constexpr detached_t detached;
```

The class `detached_t` is a completion token that is used to indicate that an asynchronous operation is detached. That is, there is no completion handler waiting to receive the operation's result. It is typically used by passing the `detached` object as the completion token argument.

This class is independent of the coroutine facility and may have some utility in other use cases.

4.7. Class `redirect_error_t` and function `redirect_error`

```
template<class CompletionToken> class redirect_error_t;

template<class CompletionToken>
  redirect_error_t<decay_t<CompletionToken>::type>
    redirect_error(CompletionToken&& completion_token, error_code& ec);
```

The class template `redirect_error_t` is a completion token that is used to specify that the error produced by an asynchronous operation is captured to an `error_code` variable. By intercepting the error code before it is passed to the coroutine, we may prevent the coroutine from throwing an exception on resumption. For example:

```
char data[1024];
std::error_code ec;
std::size_t n = co_await my_socket.async_read_some(
    net::buffer(data), redirect_error(ctx, ec));
if (ec == net::stream_errc::eof) { ... }
```

This class is independent of the coroutine facility and may have some utility in other use cases.

5. Design discussion

5.1. Using executors to coordinate multiple threads of execution

Whether an application uses coroutines or callbacks, a chain of asynchronous operations conceptually behaves as though it is a thread of execution. Furthermore, all but the most trivial networking programs will consist of multiple threads of execution interacting and operating on shared data.

Consequently, it is essential that coroutine facilities intended for networking support executors. This allows us to manage the scheduling of related coroutines that operate on shared data. Indeed, we should allow the scheduling of both coroutine- and non-coroutine-based threads of execution in a single program.

This proposal addresses this by encoding the executor properties of a thread of execution into the `basic_unsynchronized_await_context` completion token. When passed to an asynchronous operation, the operation will utilize the associated executor when resuming the coroutine.

Similarly, the await context completion token may be passed to child coroutine functions to ensure that these callees observe the same executor properties as the caller, as illustrated in the "Refactoring" example above.

5.2. Introducing new threads of execution should be explicit

As mentioned above, coordinating multiple threads of execution is a requirement of all but the most trivial applications. Even if a networking application is single-threaded, there still exists concurrency in the scheduling and execution of these threads of execution. Therefore, to reduce the risk of programmer error, the introduction of new threads of execution should be explicit.

In this proposal, new coroutine-based threads of execution are initiated using the `spawn` function. In addition to launching a new thread of execution, this function requires the programmer to specify the executor that will be used for it.

5.3. Refactoring and composition

Unlike the approach proposed in P0055R0, this proposal does not encode the implementation of an asynchronous operation into an initiating function's return type. Specifically, all asynchronous operations that participate in a coroutine return an `awaitable<T>`. This allows us to perform simple, non-coroutine based composition of coroutine-aware functions, as in:

```
awaitable<void> throttled_post(await_context ctx)
{
  if (throttle_required())
    return my_simple_timer.async_wait(ctx);
  else
    return post(ctx);
}
```

Indeed, this proposal's `awaitable<T>` return type mirrors (most of) the regular behaviour of "normal" function return types. (The main exception being a lack of convertibility between types.) This allows end users to compose asynchronous operations and coroutines alike, as shown in the "Refactoring" example above.

5.4. Placement of the `await_context` arguments

In this proposal, the `await_context` is passed as the final argument to a thread of execution's entry point. In early prototypes it was passed as the initial argument, but this interfered with the ability to implement `spawn` using `std::invoke` (necessary to support `spawn`-ing member functions).

5.5. Performance notes

This library proposal should have minimal performance overhead on top of that already imposed by the `co_await`-based coroutine mechanism.

First, the P0055R0 approach of encoding the implementation into the initiating function return type appears to be unnecessary. Instead, asynchronous operations can encapsulate "allocated" state into a temporary coroutine that is then returned by the initiating function inside an `awaitable<T>` object. The compiler's allocation/deallocation elision optimization should then eliminate the allocation. (Unfortunately, at the time of writing this could not be verified, due to lack of access to a compiler with this optimization.)

Second, in low latency scenarios where single-threaded execution is employed, use of `basic_unsynchronized_await_context` ensures that coroutines introduce no additional synchronization overhead.

What is less certain, however, is the performance impact of refactoring code into child coroutines within a thread of execution (as shown in the "Refactoring" example above). There is significant machinery required to transport a return value from a callee to the caller. It is not clear whether compiler heroics can reduce this cost to something approaching a normal function return, let alone the coroutine equivalent of inlining the callee.

6. Impact on the standard

This is a pure extension to the draft Networking Technical Specification. It does not require changes to that specification nor to any other part of the standard.

7. Relationship to other proposals

This paper proposes an extension to the draft Networking Technical Specification to add support for `co_await`-based coroutines. These coroutines are specified in P0057R1.

This paper provides an alternative design for integrating the coroutines to that proposed in P0055R0 *On Interactions Between Coroutines and Networking Library*. In particular, this proposal requires no modification to the design of the draft Networking Technical Specification, and it addresses the design issues raised in section 5 of P0162R0 *A response to P0055R0*.

8. Proposed text

TBD

9. Acknowledgements

The author would like to acknowledge Jamie Allsop and Arash Partow for providing design feedback and comments on this proposal.